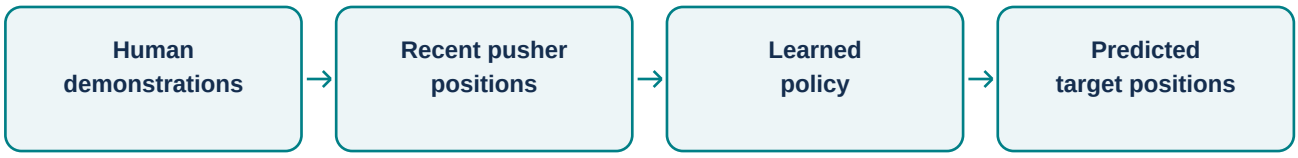
Understand your PushT notebook

A practical guide to the data, three policy networks, training settings, loss, and evaluation

Companion: PushT_Assignment_Style.ipynb
Prepared: 4 November 2026
Audience: a student learning how the complete experiment works.

What are you trying to learn?

You are learning a **policy**: a function that takes recent observations and predicts what action a human demonstrator would take. This is **behavior cloning**, a supervised form of imitation learning. The labels are continuous target positions, so this is regression rather than classification.



The most important limitation
Your parquet file provides the pusher position, but no block pose or keypoints. The current networks do not see the block or the video. They are useful starter policies, but they cannot fully observe the task your proposal describes.

Read this guide in order	Pages
Goal, dataset, and preparation	1-3
MLP, LSTM, and Transformer architectures	4-6
Settings and architecture changes	7-8
Loss and the training loop	9-10
Testing, experiment workflow, and sources	11-13

Your research question: do observation history and the choice of model improve action prediction on unseen episodes, and eventually improve closed-loop pushing? A larger network or lower training loss does not answer that question by itself.

This guide describes the supplied notebook, not a completed research study. Defaults and implementation details are taken from its actual code. Sources and validation scope appear on page 13.

02 / DATASET

What does the dataset provide?

The pinned Hugging Face **lerobot/pusht** snapshot contains **206 demonstration episodes, 25,650 frames, and a 10 Hz sampling rate**. One episode is one recorded attempt; a row is one observation/action time step. Its only declared split is train (all episodes). [1,2]

Field	Meaning and notebook use
observation.state	Two floats: pusher x and y. This is the **only model input feature**. The name does not mean full environment state.
action	Two floats: the expert's desired pusher target x and y. These are the regression labels, not velocity or a direction class. [3]
episode_index / frame_index	Identify the demonstration and frame within it. Used to split and order data; never input to a network.
timestamp / index / task_index	Time within the episode, global row identity, and task identity. Used for checks or bookkeeping, not prediction.
next.reward / next.done / next.success	Recorded outcome-related fields. Present in parquet, but excluded from inputs and the behavior-cloning loss.
observation.image	96 x 96 RGB video feature in metadata. Images are stored in a separate MP4; they are not decoded by this notebook.

The physical task and the training task

In the simulator, a circular pusher must move a T-shaped block onto a target zone. Actions specify a target position in a 512-by-512 coordinate workspace. The documented success criterion is at least 95% block coverage of the goal zone. [3]

Your current training task is narrower: given recent recorded pusher positions, predict recorded expert targets. It does not optimize the stored reward, discover actions through exploration, or prove that the block reaches the goal.

Why the missing block information matters
The same pusher position can occur with the block in different poses, requiring different actions. Pusher history can reveal motion, but cannot uniquely recover arbitrary hidden block states. This ambiguity can cause irreducible prediction error and averaged actions.

Proposal correction: the expected extra 16 block/keypoint values are absent from this parquet snapshot. The simulator supports richer observations, but that does not mean those values were saved in this dataset. [1-3]

03 / Q1 IN THE NOTEBOOK

From parquet rows to training samples

1. Verify and organize the data

The notebook pins a dataset commit, discovers every data parquet shard, and concatenates them with pandas. It checks required columns, missing/nonfinite values, duplicate frame identifiers, repeated global indices, frame continuity, and timestamp spacing. Frames are sorted by episode and frame index. A failed assertion asks you to investigate instead of silently changing temporal alignment.

2. Split whole episodes

A fixed split seed shuffles episode IDs into five outer folds. For FOLD=0, the split is **147 training / 17 validation / 42 test episodes**, corresponding to **18,233 / 2,147 / 5,270 frames**. Validation receives 10% of the non-test episodes, rounded up. Adjacent frames from one demonstration cannot appear in different sets. [1]

A random frame split would let the model train on frames almost identical to its test frames. Episode separation measures generalization to other demonstrations. It does not guarantee that all episodes have dissimilar physical starting states.

3. Fit normalization on training frames only

```
normalized value = (raw value - training mean) / training std
raw prediction = normalized prediction * action std + action mean
```

Separate means and standard deviations are fitted for the two observation and two action coordinates. A minimum standard deviation of 0.000001 avoids division by zero. Validation and test data reuse these training statistics; they never help estimate them. The repository's global statistics are not used.

4. Build a history and action chunk at time t

```
input: o[t-K+1], ..., o[t] shape [K, 2]
target: a[t], ..., a[t+H-1] shape [H, 2]
batched input / target: [B,K,2] / [B,H,2]
```

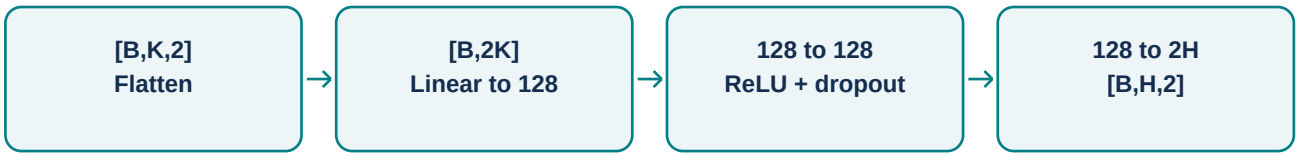
K is observation history length; **H** is the number of actions predicted together. At the beginning, repeat the first observation to fill the history. At the end, pad unavailable future targets with normalized zeros and mask them out of the loss. No window crosses an episode boundary.

Time alignment is deliberate: observations end at t and the first label is action a[t], consistent with the proposal. At 10 Hz, K=4 spans 0.3 seconds from its first to last observation; H=8 contains targets at offsets 0.0 through 0.7 seconds.

04 / Q2 - MLPPOLICY

MLP: learn a fixed-window mapping

MLP means multilayer perceptron. It joins the observations into one vector, applies two nonlinear hidden layers, and predicts the action chunk. Each history position has its own input weights, so time order is preserved by its place in the flattened vector.



Stage	What happens
Flatten	For K=4, four 2D observations become eight input numbers per sample.
Hidden layer 1	Linear(2K,128), then ReLU, then dropout. A linear layer learns weighted combinations plus biases.
Hidden layer 2	Linear(128,128), ReLU, dropout. ReLU replaces negative activations with zero, allowing nonlinear relationships.
Output head	Linear(128,2H), reshaped to [B,H,2]. No softmax or sigmoid: continuous normalized coordinates can take any value.

What it can learn

With K=1, this is a reactive current-position baseline. With K>1, it can learn patterns involving earlier and later positions, such as motion trends, through ordinary weighted connections. It has no recurrent memory beyond the observations explicitly supplied in the window.

What changing it does

Increasing the hidden width can represent more complex mappings but adds parameters and overfitting risk. Increasing K enlarges the first layer. Increasing H enlarges the output layer. With K changed, you must construct a new model because the input size changes.

```
Default K=4, H=1: 8 -> 128 -> 128 -> 2
Trainable parameters: 17,922
General count: 256*K + 16,640 + 258*H
```

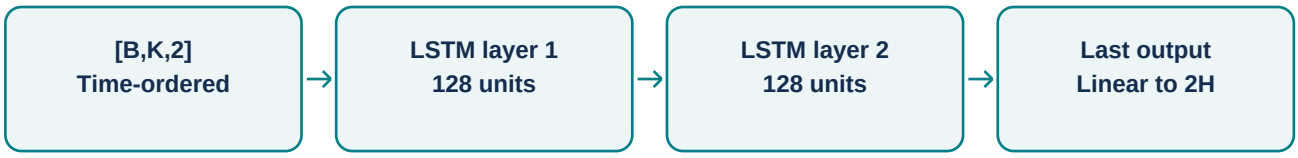
Interpretation
The MLP is a strong first check: it is simple, fast, and easy to debug. A sequence model only earns its extra complexity if held-out results improve under a fair comparison. All models in the notebook use K=4 by default; run K=1 explicitly for the proposal's current-state baseline.

Parameter counts include learned weights and biases. They do not count observations or optimizer state. Counts here are for the delivered defaults, rather than the H=8 smoke-test configuration.

05 / Q3 - LSTMPOLICY

LSTM: summarize an ordered sequence

An **LSTM** is a recurrent model. It reads positions one time step at a time and updates a hidden representation. Its gates control what information is retained, written, and exposed. This gives it a different bias for learning temporal patterns than flattening the history. [4]



What happens inside a time step?

The **cell state** acts like a running information store. A forget gate can reduce old information; an input gate controls how much candidate information is added; an output gate controls how much of the cell state appears in the hidden state. Gates are learned numerical functions, not hand-written rules about robot motion.

```
cell[t] = forget[t] * cell[t-1]
+ input_gate[t] * candidate[t]
hidden[t] = output_gate[t] * tanh(cell[t])
```

Multiplication is elementwise. Gate values are between zero and one; the candidate and tanh terms can be positive or negative. Both the current observation and previous hidden state contribute to the gate computations. [4]

Exactly what your class does

The two stacked LSTM layers each have hidden size 128 and are unidirectional. The output sequence has shape [B,K,128]. The model takes its **last output**, applies an extra dropout layer, then Linear(128,2H), and reshapes the result to [B,H,2]. LSTM dropout is also applied between the two recurrent layers. [1,4]

Memory resets for every sample
No hidden state is passed into the LSTM call, so it starts fresh for each window. It remembers within K observations; it does not carry memory through an entire episode. The inference helper uses the same sliding-window convention.

Capacity and tradeoffs

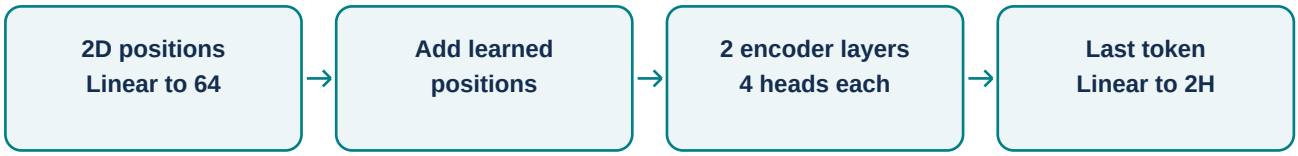
At K=4 and H=1, the LSTM has **199,938 parameters**. Changing K changes the amount of sequence computation, but not its weight count. More hidden units or layers increase capacity and cost. Longer history can help identify motion, but cannot supply block information that was never observed.

The notebook uses the last sequence output from the top layer. It does not use a bidirectional LSTM, teacher-forced previous actions, or a persistent hidden state across minibatches.

06 / Q4 - TRANSFORMERPOLICY

Transformer: compare observations with attention

A Transformer lets each observation form weighted combinations of other allowed observations. Unlike the LSTM, it does not have to compress all earlier observations into one recurrent state before comparing them.



Tokens and position information

Each 2D observation is projected into a 64-value **token**. A learned 64-value position vector is added for each slot in the K-frame window, so the model can distinguish earlier from later observations. Position means the slot within this window, not the global frame index. [1]

Attention, in plain language

A token produces a **query** describing what information it is looking for, a **key** describing what it offers, and a **value** carrying information. Query-key similarity becomes a set of attention weights; those weights combine values. Four heads allow different learned comparisons in parallel. Attention weights are learned routing, not proof of a physical explanation.

```
attention(Q,K,V) = softmax(Q*K_transpose / sqrt(d_head)) * V
d_model = 64; heads = 4; d_head = 16
```

Each encoder layer also contains a tokenwise feed-forward network of width 128, ReLU, residual connections, dropout, and layer normalization. Your code uses two such layers, then maps the last token to 2H outputs. These components refine representations and make optimization more stable. [1,5]

What the causal mask does

The triangular mask prevents a token from looking at a later history slot. The last token can still attend to the entire supplied history. No observation after t is ever an input. This is an encoder used for regression, not a language model generating one action token at a time.

Size and history length
The default has **67,522 parameters**. Learned position vectors have size [1,K,64], so changing K requires a fresh model. Attention work grows roughly with K squared per layer. At these small K values the overhead may matter more than the theoretical scaling. More heads do not automatically improve accuracy.

07 / EXPERIMENT SETTINGS CELL

Settings you can change first

Setting (default)	Effect and practical interpretation
K = 4	History length; allowed values 1,2,4,8. More context may expose motion. It also adds computation and may add distracting or ambiguous history.
H = 1	Predicted action chunk; 1 or 8. H=8 learns a longer expert sequence, increases output size, and masks targets at the episode end. It does not execute eight actions in this notebook.
LR = 0.001	Adam learning rate. Larger steps can learn faster or become unstable; smaller steps can learn slowly. If loss oscillates, try 0.0003 while keeping other settings fixed.
BATCH_SIZE = 256	Samples used for one gradient update. Larger batches use more memory and make fewer updates per epoch. Smaller batches give noisier gradients and more updates.
EPOCHS = 10	Maximum complete passes over training windows. More is a larger training budget; early stopping can finish sooner.
PATIENCE = 3	Stop after three consecutive epochs without a lower validation MSE. Larger patience tolerates plateaus; smaller patience saves time but may stop too soon.
DROPOUT = 0.1	During training, randomly suppress some activations. May reduce overfitting; too much may prevent learning. It is disabled for evaluation. Start by comparing 0.0 and 0.1. [6]
WEIGHT_DECAY = 0.0001	L2-style regularization in this Adam call. Discourages large weights. Too strong can underfit; zero removes this optimizer penalty. [7]

How should you tune them?

Change one family of choices at a time and judge progress on **validation**, not test. The numerical alternatives above are starting experiments, not guarantees. A useful first progression is K=1/H=1, then K=4/H=1, then H=8. Keep the same split and compare the same settings across all three architectures.

These are hyperparameters chosen by you. Model weights and biases are trainable parameters learned from data. Adam also has defaults such as betas=(0.9,0.999) and eps=1e-8; the notebook leaves them unchanged. [7]

08 / WHAT NEEDS MORE CARE

Split controls and architecture changes

Control	What it changes
SEED = 42	Model initialization, dropout randomness, and loader shuffle. Vary for repeated training runs. A fixed seed improves repeatability but does not guarantee identical results across devices or library versions.
SPLIT_SEED = 42	Episode partition randomness. Keep fixed across models and training seeds for a directly comparable study.
FOLD = 0 / N_FOLDS = 5	Selects the outer test fold. Run FOLD=0 through 4 to cover all five. The current assertion accepts only 0 through 4: changing N_FOLDS also requires updating that check.
VAL_FRACTION = 0.10	Fraction of non-test episodes reserved for validation. A larger value gives more validation evidence but fewer training episodes. It is not 10% of the whole dataset.
REVISION / REPO_ID	Which dataset snapshot is loaded. Changing them can alter schema or content and requires renewed inspection. Keep the revision pinned for reproducibility.
USE_GOOGLE_DRIVE / OUT	Persistence and destination of saved artifacts. These change storage, not learning. Session-local files disappear when a Colab session expires.

Changing architecture requires editing the Network cells

MLP: change hidden widths (128) and keep adjacent Linear dimensions consistent. **LSTM:** change hidden_size=128 or num_layers=2, and update the head input size if hidden size changes. With one recurrent layer, its built-in inter-layer dropout has no place to act; the separate head dropout still works. [4]

Transformer: change embedding width (64), feed-forward width (128), head count (4), or layer count (2). Embedding width must be divisible by head count. Update the embedding, position vectors, encoder, and head consistently. More width/layers usually means more parameters, memory use, and opportunity to overfit. [5]

Rerun from the settings after a change
Recreate datasets and fresh models when K, H, the fold, or the seed changes. Refit normalization whenever training episodes change. Do not load old weights with incompatible shapes. The current checkpoint reload code assumes the delivered architecture constants; if you edit them, also record and reconstruct those choices.

Keep the parameter-count check below one million if you want to retain the proposal's size limit. Similar maximum epochs are not identical compute: early stopping, batch size, model size, and runtime also affect the comparison.

09 / LOSS_PARTS

The loss function, step by step

The network predicts normalized x/y action coordinates. The loss is **masked mean squared error (MSE) per valid coordinate**. It measures how far predictions are from the expert targets. A lower value means a closer numerical match on this dataset; it is not an overlap score.

```
L_data = sum[b,j,d] mask[b,j] * (prediction[b,j,d] - target[b,j,d])^2
/ (2 * sum[b,j] mask[b,j])
```

Here **b** indexes a sample in the batch, **j** an action offset from 0 to H-1, and **d** the x or y coordinate. The mask is one for a real expert action and zero for a padded target. The denominator has a factor of two because every valid action has two coordinates.

A small worked example

Suppose H=2 but only the first action exists at the end of an episode. The valid target is [0.5, -0.5], and the prediction is [0.7, -0.2]. The second predicted action is ignored because its mask is zero.

```
mask = [1, 0]
valid errors = [0.2, 0.3]
squared errors = [0.04, 0.09]
L_data = (0.04 + 0.09) / 2 = 0.065
```

If the padded second prediction were extremely wrong, it would still contribute zero to this loss. A normalized zero padding value is a storage placeholder, not an expert instruction to move to the training mean.

What is actually optimized?

Backpropagation differentiates this data loss. Adam updates the weights, and the configured weight decay adds L2-style pressure. With the ordinary Adam call used here, the penalty is coupled to the adaptive optimizer, unlike AdamW's decoupled decay. The notebook does **not** manually add a separate penalty to the reported MSE. [7]

Consequently, training/validation/test CSV losses report prediction error alone. Gradient norm clipping at 1.0 limits large training updates; it does not clip actions or change the formula used for evaluation.

Why normalization matters
A normalized error of one means one training standard deviation in that coordinate. Original-coordinate MSE is recovered by multiplying each coordinate's squared normalized error by its own action variance. You cannot generally use one shared scale factor for both coordinates.

10 / TRAIN_POLICY AND RUN_EPOCH

What happens during training?

Step	Action in your notebook
1. Start fresh	Reset the training seed and instantiate a new network. Create Adam with the specified learning rate and weight decay. Shuffle only the training samples.
2. Predict a batch	Move inputs/targets/masks to CPU or GPU. Clear old gradients. Run the model on [B,K,2] observations to obtain [B,H,2] predictions.
3. Learn	Compute masked MSE, run backward(), clip the gradient norm to 1.0, and call optimizer.step(). Only training batches change the weights.
4. Validate	Use model.eval() and disable gradient tracking. Dropout turns off. Run the separate validation episodes without optimizer updates.
5. Keep the best	If validation MSE is lower than every earlier epoch, copy the weights. Otherwise increase the non-improvement counter. Stop when it reaches PATIENCE or EPOCHS ends.
6. Restore and save	Reload the best validation weights rather than automatically using the final epoch. Save a checkpoint and training-history CSV.

Reading the learning curves

Both losses decrease: the model is learning patterns that transfer to validation. **Training falls while validation rises:** possible overfitting; examine the best epoch and regularization. **Both remain high:** possible insufficient information, underfitting, optimization difficulty, or a data/alignment problem.

A training curve is accumulated while weights change throughout the epoch and dropout is active. Validation is computed afterward with fixed weights and dropout disabled. Training loss therefore need not always be lower than validation loss. The notebook weights epoch aggregates by valid coordinates, rather than averaging batch averages equally.

What gets saved?

Checkpoints include the model weights/type, settings, training normalization, dataset revision, episode IDs, split-manifest digest, selected epoch, validation score, library versions, and elapsed time. Separate files contain episode splits, frame assignments, normalization arrays, histories, test metrics, and run metadata.

Checkpoint loading uses weights_only=True and an explicitly chosen model name. Only your own trusted checkpoints should be loaded. Saved artifacts support reproducibility; they do not substitute for running the planned five folds and multiple seeds.

11 / EVALUATION AND RESEARCH CLAIMS

What testing tells you

Set or metric	Meaning
Training episodes	Used to fit weights and normalization. Performance here measures fit to examples already used for learning.
Validation episodes	Used for checkpoint selection and hyperparameter decisions. They are not a final unbiased test after repeated tuning.
Test episodes	Held out from fitting and checkpoint selection. Use for final reporting after fixing choices. In Q2-Q4, report_test evaluates each restored model.
normalized_mse	Total squared normalized error divided by all valid action coordinates. Useful for the training objective; different folds have different normalization scales.
raw_coordinate_mse	Total squared error in original x/y coordinate units divided by valid coordinates. Units are coordinate-units squared.
episode_mean_raw_mse	Compute raw MSE for each episode, then average episodes equally. A short episode and a long episode each receive one equal vote.
Train-mean baseline	Always predicts the mean training action. Zero in normalized action space. A learned policy should be compared against this simple baseline.

Horizon plots and overlapping chunks

The plot groups offline error by action offset. At H=1 there is only one point. At H=8, later offsets contain fewer valid samples near episode ends. Overlapping chunks can count a recorded action several times as a target, but only within its assigned episode/split. These are chunk-window metrics, not one unique prediction per recorded action.

Offline accuracy and closed-loop success are different

The current test supplies **expert-recorded observations**. A real rollout feeds the policy observations caused by its own earlier actions; small mistakes can change later states. That distribution shift is why low offline MSE does not establish successful pushing or measure compounding error.

The proposal's later simulator test should use matching observations/actions, fresh starts, and one consistent rollout budget. Report coverage/overlap and success across 50 evaluation episodes, with videos and seed variability. Execute the first predicted action and replan. The present notebook contains no such rollout implementation or results. [1,3]

12 / PRACTICAL WORKFLOW

How to run a useful experiment

Follow the notebook in this order

Setup: optionally enable Drive, install dependencies, run imports/helpers, then set the configuration. **Q1:** download, inspect, split, normalize, create windows and loaders. **Q2:** MLP network, training, curves, test. **Q3:** LSTM. **Q4:** Transformer. **Q5:** compare metrics and curves, reload a checkpoint, and demonstrate predictions.

A staged study

Stage	Question answered
Baseline: K=1, H=1	How well does each model predict from only the current pusher position? Does it beat the train-mean predictor?
History: K=2,4,8; H=1	Does additional pusher history improve validation error? Compare architectures at the same K to isolate architecture effects.
Chunks: H=8	Does predicting several expert actions jointly change performance? Compare offset-zero errors separately from whole-chunk MSE when contrasting H=1 with H=8.
Regularization	At a fixed K/H, compare dropout 0.0 vs 0.1 and inspect validation curves. Hold other choices fixed.
Replicates and folds	Run training seeds 0,1,2 with the same split seed. Repeat FOLD=0...4, fitting new statistics each time. Report mean and variability.
Richer observations	Add verified block/keypoint features or synchronized image inputs before claiming to test the proposal's fully observed policy design. Then implement simulator evaluation.

The complete proposed grid is **3 architectures x 4 histories x 2 horizons x 2 dropout settings x 3 seeds x 5 folds = 720 training runs**. Establish a small reliable run before launching it. Training seeds measure optimizer variability; folds measure variation across held-out episode groups. These are different sources of uncertainty.

Keep the outer test meaningful
Do not pick K, H, dropout, or architecture by repeatedly checking the same outer-test results. Choose with validation inside each fold or predefine the comparison. Repeating five folds does not make repeated test-set tuning harmless.

Output names include model/fold/seed/K/H/dropout, but not every training setting. Split and normalization filenames are partly shared. Use a separate OUT directory for each experiment when changing learning rate, width, split seed, or other settings to avoid overwriting files.

13 / REFERENCE

Glossary, evidence, and sources

Term	Meaning in this project
Policy / behavior cloning	A learned observation-to-action function / fitting that function to demonstrated actions.
Epoch / batch	One pass over training windows / the samples used for one update.
Overfitting / underfitting	Learning training-specific patterns that transfer poorly / failing to capture useful structure.
Checkpoint / inference	Saved model state / using a trained model to predict, without updating it.
Ablation / fold	A controlled change to study one component / one partition in cross-validation.

What has actually been verified?

The notebook schema, code syntax, and actual pinned parquet schema were checked. The complete notebook workflow was executed locally on all 25,650 rows with **K=4, H=8, one epoch per model**, including validation, offline testing, saving, and checkpoint reload. Forward/backward checks passed for all proposed K/H combinations. The delivered default is K=4, H=1, up to 10 epochs. The smoke test demonstrates execution, not convergence or an architecture ranking. The full grid and simulator tests have not been run.

Source notes

[1] Project artifacts: PushT_Assignment_Style.ipynb (the companion notebook) and the local EECE568_Proposal.pdf, *Comparing MLP, LSTM, and Transformer Policies for Robot Imitation Learning on PushT*, Maryam Sharifzadeh and Navraj Kambo, UBC EECE 568, pp. 1-2. The guide describes the actual code where it differs from the proposal.

[2] Hugging Face lerobot/pusht - pinned metadata and dataset snapshot

Revision: 7628202a2180972f291ba1bc6723834921e72c19. Metadata and the downloaded parquet were inspected directly. Metadata is more current than the dataset card's older v2 example.

[3] gym-pusht - task, observation modes, action space, and success criterion

[4] PyTorch 2.14 - LSTM

[5] PyTorch 2.14 - TransformerEncoderLayer

[6] PyTorch 2.14 - Dropout

[7] PyTorch 2.14 - Adam

Online references checked 4 November 2026. The linked PyTorch version matches the local execution version; a future Colab runtime may provide a different version. Conceptual explanations and suggested experiment choices are instructional interpretation, not measured performance claims.